Primeira Avaliação Parcial

Linguagens de Programação

Prof. Lucas Ismaily

Aluno:

Informações importantes:

- A prova vale 11 pontos;
- Caso não saiba responder uma questão, se você escrever "não sei" ganhará 10% da questão. Por exemplo, se uma questão vale 2 pontos, você receberá 0, 2 décimos;
- **Prova individual**. Sejam honestos com vocês e comigo, por favor. Se for detectado qualquer tipo de fraude, os envolvidos receberão nota zero. Note: os envolvidos receberão zero, não importa se você foi a origem ou o destino, ambos receberão nota zero.

Questões:

1. (3,0 pontos) **Estudo de caso: O Paradoxo do Código Perfeito.**

   No final de 2024, uma startup de pagamentos, precisava reescrever do zero o seu motor de análise de fraudes. O sistema antigo, escrito em uma linguagem antiga e com baixo suporte à abstração, havia se tornado um pesadelo de manutenção.

   Para o novo projeto, a equipe técnica foi dividida. De um lado estava Roberto, um arquiteto de software veterano. Do outro, Marina, uma desenvolvedora sênior brilhante, recém-chegada com forte background em matemática e linguagens funcionais puras.

   A equipe precisava escolher a linguagem ideal para o novo motor, pesando os custos: eles precisavam de algo com compilação rápida, execução de alta performance, mas também baixo custo de treinamento, já que a empresa planejava contratar dezenas de desenvolvedores juniores no ano seguinte. Roberto sugeriu Golang, argumentando a favor da sua simplicidade e facilidade de leitura. Marina defendeu apaixonadamente o Scala, focando na sua gigantesca expressividade e poderoso sistema de tipos.

   No fim, Marina convenceu a diretoria a usar Scala. Para provar seu ponto, ela pegou um algoritmo complexo de detecção de anomalias financeiras que, no sistema antigo,

ocupava 300 linhas de código confuso. Usando recursos avançados da linguagem, como monads, conversões implícitas e funções de alta ordem, ela reescreveu o algoritmo inteiro em apenas 4 linhas de código denso e elegantíssimo.

O motor entrou em produção e rodou perfeitamente por três meses. Era rápido e o código base era minúsculo. No entanto, em uma sexta-feira de feriado prolongado, enquanto Marina estava em um acampamento sem sinal de internet, uma nova regra do Banco Central exigiu uma mudança urgente na lógica de fraudes. O sistema começou a bloquear transações legítimas, gerando prejuízos de milhares de reais por minuto.

O único desenvolvedor de plantão era Lucas, um programador pleno que havia acabado de ser treinado. Quando Lucas abriu o arquivo de Marina, ele se deparou com aquelas 4 linhas altamente expressivas. Como a linguagem permitia que Marina criasse seus próprios operadores e ocultasse detalhes em abstrações complexas, o código parecia hieróglifos para ele.

Nesse momento, a altíssima expressividade da linguagem destruiu completamente a simplicidade. O que Marina resolvia mentalmente com facilidade, exigia que um desenvolvedor comum passasse horas desvendando a documentação para entender como os dados estavam fluindo ali.

O custo de manutenção e execução daquele erro explodiu. A equipe levou 10 horas para entender as 4 linhas de código, fazer a alteração e subir a correção, algo que levaria 15 minutos em uma linguagem menos expressiva, porém mais verbosa e simples.

Com base no relato, responda à seguinte questão: analise criticamente como os critérios de legibilidade, expressividade e custo (de treinamento, compilação, execução e manutenção) determinaram o sucesso inicial e a falha posterior do projeto na adoção da linguagem escolhida. Em seguida, utilizando o evento ocorrido no feriado prolongado, descreva com suas próprias palavras por que, no design de linguagens, a busca por uma altíssima expressividade frequentemente entra em conflito direto com a legibilidade.

2. (2,0 pontos) Converta as seguintes expressões regulares para autômatos finitos não determinísticos.

   (a) $((a^*|b)^*|(ca)^*)$

   (b) $(a|\epsilon)^*b$

3. (3,0 pontos) No processo de compilação, diferentes estratégias de análise sintática podem ser utilizadas, como os métodos baseados em gramáticas LL ou LR. Embora ambos tenham o objetivo de reconhecer cadeias válidas de uma linguagem, eles apresentam características distintas. Por que, em muitos compiladores reais, métodos LR são preferidos em relação aos métodos LL, mesmo sendo mais complexos de implementar?

4. (3,0 pontos) Considere a seguinte gramática:

0. $S' \to S\$$
1. $S \to AaAb$
2. $S \to BbBa$
3. $A \to \epsilon$
4. $B \to \epsilon$

(a) A gramática é $LL(0)$? Dê evidência.
(b) A gramática é $LL(1)$? Dê evidência.
(c) A gramática é SLR? Dê evidência.